

Annual Cleanup Runbook

The June sequence, start to finish · for the next RDS co-op

This is the full June sequence for the ArcGIS Online cleanup, written for whoever holds the RDS co-op seat and can run R and Python. Work from the repository root. Two rules sit above everything else: **real data never leaves institutional storage** (it goes in the git-ignored data/ folder while you work), and **the dry run always comes before the live run**.

The list this builds is a *candidate*. Graduation is confirmed by IT on the returned sheet (step 3), and you should expect to do a little R wrangling to reconcile what comes back — the deletion script runs on the returned sheet, not the raw candidate.

0 · Prerequisites

- R with `tidyverse` and `lubridate` (for the identification step).
- Python 3 with the ArcGIS API: `pip install arcgis`.
- An **org-administrator** AGO account (deleting other users' content needs it).
- The repository, cloned from the library/RDS institutional account.

The admin password is never stored — the deletion script prompts for it at runtime. Don't put it in any file.

1 · Download the reports

From AGO (Organization → Reports), export into data/:

- the **item report** (item activity — owner, storage sizes, share level, view counts, date last viewed, recycle-bin flag);
- the **member report** (Username, Email, Role, User Type, Member Categories, Last Login Date, account status).

Also confirm `data/keep_list.csv` is current — the never-delete list of co-ops and protected staff. Both scripts read it and protect anyone on it.

2 · Build the candidate list

```
Rscript scripts/build_priority_list.R
```

This writes `data/outputs/cleanup_priority.csv`. Read the three checks it prints before trusting it — the units total (~1,151 GB), the faculty value lists, and the recoverable total (~557 GB). Full detail is in *Building the Priority List*.

3 · The IT round-trip

Graduation is IT's data, not ours. Send them the candidate list; they stamp the graduation status onto it and hand it back. **Save the returned sheet as** `data/cleanup_priority.csv` — **that's the deletion script's input**. It's the same round-trip the program has always used, just on a list the script now builds for you.

The deletion script acts only on rows where `status = DEPARTED`. That value comes from IT's returned sheet, so a row is never deleted on our say-so alone.

4 · Notify the owners

Email the flagged owners and record the send date in each row's `notice_date` column. This starts the 15-day grace clock; the deletion script won't act on a row until that window has elapsed.

5 · Dry run

Confirm `DRY_RUN = True` in `scripts/cleanup_targets.py`, then:

```
python3 scripts/cleanup_targets.py
```

Read `data/outputs/cleanup_manifest.csv` (everything considered, with its disposition) and `data/outputs/cleanup_review_needed.csv` (public items and anything with a dependent — held back on purpose).

6 · Review and authorize

A second person reads the manifest and the review file and signs off. Spot-check that nothing owned by an active or faculty user sits in the delete set. This sign-off is policy, not a formality.

7 · Live run

Set `DRY_RUN = False` and leave `PERMANENT_DELETE = False`. For an unattended Notebook run, set `REQUIRE_CONFIRMATION = False`; interactively, leave it on and type the confirmation.

```
python3 scripts/cleanup_targets.py
```

Each in-scope item is exported to `data/outputs/backups/`, dependency-checked, then deleted to the 14-day recycle bin. Items whose backup fails are skipped, not deleted.

8 · Close out

- Promote `data/outputs/cleanup_priority_updated.csv` (with the deleted-flag write-back) to next year's tracking sheet.
- Archive `data/outputs/cleanup_manifest.csv` and `data/outputs/cleanup_results.csv` to the audit trail.

- Move data/outputs/backups/ to institutional storage and keep per the retention policy.

Troubleshooting

- **“Nothing in scope.”** The column names in the CSV don't match what the script expects — check `classify_row()`.
- **503 errors mid-run.** You're hitting the API too fast; raise `THROTTLE_SECONDS`.
- **A map broke after a run.** Find the item in `data/outputs/cleanup_results.csv` and restore it from `data/outputs/backups/` (or the 14-day recycle bin).

Cadence: once a year, in June (post-commencement), on an ArcGIS Online Notebook.